

Roadmap

Phase 1 - Vektorsuche

- 1 Frontend-Zielbild
- 2 Requirement
- 3 Suchinhalt festlegen
- 4 Domain Model dokumentieren
- 5 Embedding-Modell ausprobieren
- 6 pgvector aktivieren
- 7 embedding-Spalte technisch anlegen
- 8 Embeddings erzeugen + speichern
- 9 Vektorsuche direkt testen
- 10 Backend / SQLAlchemy-Suche
- 11 Search-API
- 12 Frontend anbinden
- 13 Tests + Review

Plan

Phase 1 – Semantische Vektorsuche (ohne LLM)

Ziel: Acts nicht nur über exakte Begriffe finden, sondern nach Bedeutung (z. B. „ruhige Musik am Abend“). Ergebnis ist eine nach Ähnlichkeit sortierte Liste passender Acts.

Suchanfrage → Embedding-Modell → Query-Vektor → PostgreSQL/pgvector → passende Acts

Geplant: durchsuchbare Textfelder im Domain Model festlegen, pgvector in PostgreSQL aktivieren, Embeddings erzeugen und speichern, Ähnlichkeitssuche im Backend als FastAPI-Endpoint bereitstellen und im Frontend einbinden – jeweils mit Tests und Review.

****Phase 2 – LLM Integration**

Die Treffer aus Phase 1 werden einem LLM als Kontext übergeben, das daraus eine Antwort auf Fragen in natürlicher Sprache formuliert.

Suchanfrage → Vektorsuche → passende Acts → LLM-Kontext → generierte Antwort

Die Vektorsuche bleibt dabei die Retrieval-Schicht; das LLM ersetzt die Datenbanksuche nicht und darf keine Festivalinformationen erfinden, die nicht in den gefundenen Daten stehen.

Output

Phase 1 – Vector Search / Retrieval Layer

Nutzer fragt:

„Ich suche heute Abend etwas Ruhiges, eher atmosphärisch.“

Die Anwendung macht daraus ein Embedding und findet ähnliche Acts.

Das Ergebnis könnte direkt so aussehen:

1. Neon Echo
Genre: Ambient Electronic
Bühne: Lake Stage
Zeit: 20:30
Distanz: 0.08
2. Luna Fields
Genre: Indie Folk
Bühne: Forest Stage
Zeit: 21:00
Distanz: 0.14
3. Dream Waves
Genre: Dream Pop
Bühne: Main Stage
Zeit: 22:00
Distanz: 0.19

Das System sagt also im Kern nur:

„Diese Datensätze passen semantisch am besten zu deiner Anfrage.“

Keine eigentliche sprachliche Interpretation oder Empfehlung durch ein LLM.

Phase 2 – LLM Integration (Generation Layer)

Die gleichen drei Treffer werden jetzt zusätzlich an ein LLM gegeben, zusammen mit der ursprünglichen Frage.

Das Ergebnis könnte dann sein:

„Wenn du etwas Ruhiges und Atmosphärisches suchst, ist Neon Echo vermutlich die passendste Wahl. Luna Fields ist akustischer und weniger elektronisch, während Dream Waves etwas poppiger klingt. Wenn du möglichst entspannt starten möchtest, würde ich mit Neon Echo um 20:30 beginnen.“

Jetzt passiert mehr:

Nutzerfrage → Retrieval → gefundene Daten → LLM → formulierte Antwort

Der entscheidende Unterschied ist also:

Phase 1 liefert Treffer.

Phase 2 erklärt, kombiniert und formuliert auf Basis dieser Treffer.

oder einfach:

Phase 1: "Hier sind passende Acts."

Phase 2: "Hier sind passende Acts – und so passen sie zu deiner Frage."

Plan

Phase 1 – Semantische Vektorsuche ohne LLM

Ziel:

 **Note**

Suchtext

↓

Embedding-Modell

↓

Query-Vektor

↓
PostgreSQL + pgvector
↓
ähnlichste Artists
↓
zugehörige Acts
↓
Trefferliste

Beispiel:

Der Festival Planner bekommt eine natürliche Suche wie:

„Ich suche ruhige elektronische Musik am Abend.“

Die Anwendung liefert dazu passende Acts. Noch **kein LLM**, sondern reine semantische Suche.

Ergebnis von Phase 1:

Eine Trefferliste mit passenden Acts, z. B. Neon Echo, Luna Fields, Dream Waves.

Entwicklungsreihenfolge

1. Frontend-Zielbild bauen

- Suchfeld ergänzen
- Button „Suchen“
- Bereich für Suchergebnisse
- zunächst mit Mock-Daten
- Beispielquery sichtbar machen: „ruhige elektronische Musik“

Ziel: Kunde/Stakeholder sollen sehen, wo die Reise hingeht.

2. Requirement „Semantische Suche“ ergänzen

Zum Beispiel:

M7 – Semantische Suche:

Nutzer können Acts über natürlich formulierte Suchanfragen finden.

Die Ergebnisse werden nach semantischer Ähnlichkeit sortiert.

Noch kein LLM.

3. Festlegen, was semantisch durchsucht wird

Aktuell reichen `Artist`, `Stage`, `starts\_at`, `ends\_at` noch nicht für gute semantische Suche.

Deshalb Suchinformationen ergänzen, etwa:

```

Artist

- name
- genre
- description

```

Für das Embedding wird daraus z. B.:

```

Neon Echo. Genre: Electronic Ambient.

Atmosphärische elektronische Musik mit ruhigen Synthesizer-Flächen.

```

4. Domain Model erweitern

Sinnvoll wäre zunächst:

```

Artist

- └ id
- └ name
- └ genre
- └ description

```

Dabei bewusst klären:

> Was ist fachlicher Inhalt und was ist nur technische Repräsentation?

`genre` und `description` sind fachliche Daten.

5. Embedding-Modell separat ausprobieren

Embedding Modell installieren

Embedding-Modell: lokal

`sentence-transformers/all-MiniLM-L6-v2` , **384 Dimensionen**. Keine API-Kosten, keine Keys, und die Embedding-Erzeugung bleibt sichtbar und nachvollziehbar.

```
pip install sentence-transformers
```

Bei `sentence-transformers/all-MiniLM-L6-v2` liefert das Modell standardmäßig immer Vektoren der Länge: 384 -

Andere Embedding-Modelle haben andere Dimensionen, zum Beispiel 768, 1024 oder noch mehr.

Info

Theoretisch könntest du einen 384-dimensionalen Vektor **nachträglich reduzieren**, etwa mit PCA auf 100 Dimensionen. Dann veränderst du aber die Repräsentation und musst dieselbe Transformation konsistent für alle gespeicherten Texte und Suchanfragen anwenden!

6. pgvector in PostgreSQL aktivieren

```
```  
CREATE EXTENSION IF NOT EXISTS vector;
```
```

Danach bekommt das entsprechende Modell ein Vector-Feld, z. B. bei MiniLM:

```
```  
vector(384)
```
```

Danach kennt PostgreSQL zusätzlich unter anderem den Typ: *vector*

5. Embedding-Modell separat ausprobieren

Bevor es in die App kommt, ein kleines Experiment:

```
Text  
↓  
Tokenizer  
↓  
Embedding-Modell
```

↓

Vektor mit 384 Zahlen

Modell beispielsweise:

```
sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2
```

Damit wird sichtbar, dass das Embedding-Modell kein Textsuchsystem ist, sondern Text in einen Zahlenraum überführt.

7. `Artist.embedding` als `vector(384)` in DB/SQLAlchemy ergänzen

8. Embeddings für Festival-Daten erzeugen

Beim Seed oder in einem separaten Schritt:

```
...  
Artist + Genre + Description  
    ↓  
Embedding-Modell  
    ↓  
Vektor  
    ↓  
PostgreSQL  
...
```

Danach enthält jeder relevante Datensatz zusätzlich sein Embedding.

9. Vektorsuche zunächst direkt testen

Eine Suchanfrage wie:

```
...  
"ruhige elektronische Musik"  
...
```

wird ebenfalls eingebettet.

Dann:

```
...  
Query  
    ↓  
Query-Embedding
```

```
↓
Vergleich mit gespeicherten Embeddings
↓
kleinste Distanz
↓
beste Treffer
```
```

Zum Beispiel:

```
```
Neon Echo      0.08
Dream Waves    0.15
Iron Wolves    0.71
```
```

## 9. Vektorsuche in SQLAlchemy integrieren

Die Suche wandert in die bestehende Architektur:

```
```
routers.py
  ↓
crud.py
  ↓
SQLAlchemy
  ↓
PostgreSQL + pgvector
```
```

Zum Beispiel fachlich als:

```
```
semantic_search(query_embedding, limit=3)
```
```

## 10. Search-API ergänzen

Etwa:

```
```
GET /api/search?q=ruhige elektronische musik
```
```

Intern:

```

Suchtext

↓

Embedding

↓

pgvector

↓

Top-k Acts

↓

JSON

```

## 11. Frontend mit echter Suche verbinden

Jetzt ersetzt die reale API die Mock-Daten.

Ergebnis:

```

Suche:

"ruhige elektronische Musik"

Neon Echo

Ambient Electronic

Lake Stage · 20:30

Dream Waves

Dream Pop

Forest Stage · 21:00

```

## 12. Tests und Review

Prüfen:

- Embeddings werden erzeugt
- Vector-Feld ist korrekt
- Suchendpoint funktioniert
- Top-k wird eingehalten
- leere Query wird behandelt

- Suchergebnisse sind stabil
- bestehende Festival-Funktionen bleiben intakt